



TRAINING PROGRAMME

Python Programming Fundamentals

Write clear, tested Python programs from the first line of code

27 HOURS

FOR Beginners, students and professionals learning to program with Python



Mohamed Aziz BEN HAHHA

mohamedazizbenhaha.netlify.app

Overview

This programme teaches participants to program in Python from the very beginning. They set up a working environment, then learn variables and types, conditions and loops, lists, dictionaries and other data structures, functions, modules and packages, virtual environments and their first unit tests with pytest. They read and write files, handle errors, work with JSON and CSV data, and organise code with classes and dataclasses. Every chapter ends with exercises at the keyboard, and the programme closes with a final project in which each participant builds, tests and presents a command-line application.

At a glance

#	PHASE	FOCUS	HOURS
01	Python Basics	Setup, variables, types, operators, input and output, conditions and loops	6 h
02	Data Structures & Functions	Lists, tuples, dictionaries, sets, comprehensions, functions, modules, pytest	6 h
03	Files, Errors & Objects	File input and output, exceptions, JSON and CSV, classes, inheritance, dataclasses	6 h
04	Final Project	A tested command-line application, presented to the group	6 h
	Buffer	Catch-up, revision, extra lab time	3 h
	Total		27 h

Key facts

AUDIENCE	Beginners, students and professionals learning to program with Python
PREREQUISITES	None. Basic computer use; no programming experience needed
DURATION	27 h: 24 h of chapters + 3 h buffer
FORMAT	Short lectures with live coding, each followed by guided exercises
LAB ENVIRONMENT	Runs locally: one laptop per participant with Python 3, VS Code and Git, installed in the first chapter; no account or cost
ASSESSMENT	Exercises validated at the end of each phase and a final project presented to the group; certificate of completion for participants who complete both
OUTCOME	Participants can write, organise and test Python programs that use data structures, functions, files, error handling and classes

PHASE 01

6 H

Python Basics

Participants install Python and VS Code, write their first programs with variables, types, operators and input and output, then control the flow of a program with conditions and loops. Validated exercise: a number-guessing game that checks each guess is within range and counts the attempts.

#	CHAPTER	CONTENT	FORMAT	HOURS
1.1	Getting Started	Setup, variables, types, operators, input and output	Lecture + Lab	3 h
1.2	Control Flow	Conditions, loops	Lecture + Lab	3 h

Data Structures & Functions

Participants store and transform data with lists, tuples, dictionaries, sets and comprehensions, then split their code into functions, modules and packages, manage dependencies with venv and pip, and write their first unit tests with pytest. Validated exercise: a module of text-statistics functions with passing pytest tests.

#	CHAPTER	CONTENT	FORMAT	HOURS
2.1	Data Structures	Lists, tuples, dictionaries, sets, comprehensions	Lecture + Lab	3 h
2.2	Functions & Modules	Functions, scope, modules, packages, venv and pip, first unit tests with pytest	Lecture + Lab	3 h

PHASE 03

6 H

Files, Errors & Objects

Participants read and write files, handle errors with exceptions, exchange data as JSON and CSV, then model their programs with classes, inheritance and dataclasses. Validated exercise: a contact book stored in a JSON file, with classes for its entries and graceful handling of a missing or corrupted file.

#	CHAPTER	CONTENT	FORMAT	HOURS
3.1	Files & Errors	File input and output, exceptions, JSON and CSV	Lecture + Lab	3 h
3.2	Object-Oriented Python	Classes, inheritance, dataclasses	Lecture + Lab	3 h

PHASE 04

6 H

Final Project

Each participant builds a command-line application from a short brief, such as a task manager or an expense tracker, reusing the exercises of the phases, then presents it to the group.

#	CHAPTER	CONTENT	FORMAT	HOURS
4.1	Brief & Build	Choose the brief, design the modules and classes, write the application with file storage, command-line arguments with argparse	Project	3 h
4.2	Test & Document	pytest tests for the main functions, error handling, README	Project	1.5 h
4.3	Demo & Review	Live demo and code walkthrough; questions and feedback	Project	1.5 h

FINAL PROJECT

Command-Line Application

A command-line application, such as a task manager or an expense tracker, organised in modules and classes, storing its data in files, handling errors and covered by pytest tests.

STACK Python 3, standard library (json, csv, argparse, dataclasses), pytest, Git, VS Code

DELIVERABLE Git repository with code, tests and README, live demo